

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_2ae09dfb-e3e\agent-a6286be30d1df2e5a.jsonl`  
- SHA-256: `6f43f05cb1d62705b864b4d4106c1e0ab82b6dcb975437cb8335311510a2bcb8`  
- Source modified: `2026-06-13T19:58:37+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `wf_2ae09dfb-e3e`  
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-13T19:55:36.016Z)

Design the OFFLINE extraction of PERSON features for the 6 golden videos (the GPU step of P3). The manifest output/human_lovo_manifest.json has per-video {name, trajectory(CSV), fps, frame_width, labels(CSV)} but NO video path; the golden video files live at "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden Labelled/<Name>.MP4" (read the manifest + ls that dir to map name->file). Design a driver that, per golden video: builds shuttle candidate windows from the trajectory CSV (reuse distill_local.build_candidates / TrackNetRunner.trajectory_to_action_windows), then for each window runs PersonDetector.detections_per_frame(video, start_frame, end_frame, frame_skip) over the ORIGINAL video frame range and fusion_features.compute_person_features(...) to get the ~5 person features, labels y via training.label_candidates vs the golden CSV. Output a per-video labeled dataset (shuttle+person features) for the harness. Specify the manifest extension (add video paths) and the name->file mapping. Read fusion_features.py + person_detector.py + tracknet_runner.py. Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. Read the ACTUAL files, quote file:line. The fusion P2 segmenter (FusionSegmenter), fusion_features.py, and PersonDetector.detections_per_frame are ALREADY on master. The experiment harness (backend/eval/experiment.py: Variant/compare/compare_unlabeled/load_video_candidates) is shipped. Be precise.

2. assistant (2026-06-13T19:55:37.819Z)

I'll start by reading the manifest file and exploring the directory structure to understand the data layout, then examine the key source files to design the offline feature extraction pipeline.

3. user (2026-06-13T19:55:38.123Z)

```
1      [  
2      {  
3          "name": "adarsh_avi_singles",  
4          "trajectory": "C:/Users/avidu/Projects/Annotation  
Setup/Collect/Trajectories/adarsh_avi_full_traj.csv",  
5          "fps": 59.94, "frame_width": 1920,  
6          "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden Labelled/Adarsh  
and Avi.rallies.csv"  
7      },  
8      {  
9          "name": "kushagra_singles",  
10         "trajectory": "C:/Users/avidu/Projects/Annotation  
Setup/Collect/Trajectories/kushagra_traj.csv",  
11         "fps": 59.94, "frame_width": 1920,  
12         "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden  
Labelled/KushagraYashAviFirstVideo.rallies.csv"  
13     },  
14     ]
```

```

15         "name": "largetest_doubles",
16         "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/largetestvideo_traj.csv",
17         "fps": 29.97, "frame_width": 1920,
18         "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/LargeTestVideo.rallies.csv"
19     },
20     {
21         "name": "gbaaddy",
22         "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/gbaaddy_traj.csv",
23         "fps": 59.94, "frame_width": 1920,
24         "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/gBaaddy20May2026.rallies.csv"
25     },
26     {
27         "name": "testlarge_short",
28         "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/testlarge_traj.csv",
29         "fps": 29.97, "frame_width": 1920,
30         "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/TestLargeVideo.rallies.csv"
31     },
32     {
33         "name": "mahadevpura_singles",
34         "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/mahadevpura_traj.csv",
35         "fps": 29.97, "frame_width": 1920,
36         "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/MahadevpuraSingles.rallies.csv"
37     }
38 ]
39

```

4. user (2026-06-13T19:55:38.449Z)

```

Adarsh and Avi.MP4
Adarsh and Avi.rallies.csv
KushagraYashAviFirstVideo.MP4
KushagraYashAviFirstVideo.rallies.csv
KushagraYashAviFirstVideo.report.json
KushagraYashAviFirstVideo.report.md
KushagraYashAviFirstVideo.summary.md
LargeTestVideo.MP4
LargeTestVideo.rallies.csv
MahadevpuraSingles.MP4
MahadevpuraSingles.rallies.csv
MahadevpuraSingles.report.json
MahadevpuraSingles.report.md
MahadevpuraSingles.summary.md
TestLargeVideo.MP4
TestLargeVideo.rallies.csv
gBaaddy20May2026.MP4
gBaaddy20May2026.rallies.csv

```

5. assistant (2026-06-13T19:55:40.108Z)

Now let me examine the key source files to understand the feature extraction pipeline:

6. user (2026-06-13T19:55:40.480Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\badminton-highlight-indexer.

7. user (2026-06-13T19:55:41.094Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\badminton-highlight-indexer.

8. user (2026-06-13T19:55:41.252Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\badminton-highlight-indexer.

9. assistant (2026-06-13T19:55:42.758Z)

Let me find these files in the repo:

10. user (2026-06-13T19:55:42.848Z)

```
backend\pipeline\detectors\fusion_features.py
tests\test_fusion_features.py
tests\__pycache__\test_fusion_features.cpython-313-pytest-9.0.3.pyc
backend\pipeline\detectors\__pycache__\fusion_features.cpython-313.pyc
```

11. user (2026-06-13T19:55:43.060Z)

```
tests\__pycache__\test_person_detector.cpython-313-pytest-9.0.3.pyc
backend\pipeline\detectors\person_detector.py
tests\test_person_detector.py
backend\pipeline\detectors\__pycache__\person_detector.cpython-313.pyc
```

12. user (2026-06-13T19:55:43.109Z)

```
backend\detectors\__pycache__\tracknet_runner.cpython-313.pyc
tests\test_tracknet_runner.py
tests\__pycache__\test_tracknet_runner.cpython-313-pytest-9.0.3.pyc
tests\__pycache__\test_tracknet_runner.cpython-313.pyc
backend\pipeline\detectors\tracknet_runner.py
backend\pipeline\detectors\__pycache__\tracknet_runner.cpython-313.pyc
```

13. assistant (2026-06-13T19:55:44.165Z)

Now let me read the actual source files:

14. user (2026-06-13T19:55:44.387Z)

```
1      """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3      NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4      math is unit-testable in a GPU-free dev container. The detection (person boxes,
5      shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6      module only turns those signals into a handful of scale/translation-invariant
7      features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8      memorise this court/camera and fail to generalise off a small labelled set).
9
10     Conventions:
11     - A person box is ``(x1, y1, x2, y2)`` in pixels; ``(track_id, box)`` when motion
needs it.
12     - ``per_frame`` is ``{frame_idx: [(track_id, box), ...]}`` keyed by the ORIGINAL video
frame
13     index (so it aligns directly with the shuttle trajectory's ``.frame`` ? no clip re-
timing).
14     - A shuttle point has ``.frame``, ``.visible``, ``.x``, ``.y`` (pixels) ? the WASB
trajectory shape.
15     - ``court_polygon`` is normalised 0-1 ``[(x, y), ...]`` or ``None`` (None ? no mask,
every
```

```

16     detection counts ? the player-count-only mode).
17     - ``net`` is ``(axis 'x'|'y', position 0-1 normalised)`` or ``None`` (None ? two-sided =
0.0).
18
19     Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20     floats in [0, 1] except ``mean_player_motion`` (a non-negative normalised speed).
21     """
22     from __future__ import annotations
23
24     from typing import Dict, List, Optional, Sequence, Tuple
25
26     BBox = Tuple[float, float, float, float]
27     Net = Tuple[str, float]
28
29
30     def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31         """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32         (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33         if polygon is None or len(polygon) < 3:
34             return False
35         x, y = point
36         inside = False
37         n = len(polygon)
38         j = n - 1
39         for i in range(n):
40             xi, yi = polygon[i]
41             xj, yj = polygon[j]
42             # Does the horizontal ray at y cross edge (i, j)?
43             if (yi > y) != (yj > y):
44                 x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45                 if x < x_cross:
46                     inside = not inside
47             j = i
48         return inside
49
50
51     def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52         """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53         the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54         if frame_width <= 0 or frame_height <= 0:
55             return (0.0, 0.0)
56         x1, y1, x2, y2 = box
57         return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60     def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61         if frame_width <= 0 or frame_height <= 0:
62             return (0.0, 0.0)
63         x1, y1, x2, y2 = box
64         return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67     def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                        court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69         """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70         True (count everyone) when it is None. Unusable frame dims (w/h <= 0) make the foot
71         point meaningless, so a masked check returns False rather than testing a bogus
(0,0)."""
72         if court_polygon is None:
73             return True
74         if frame_width <= 0 or frame_height <= 0:
75             return False

```

```

76         return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
77
78
79     def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
80                         frame_height: float,
81                         court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
82         """Per-frame count of in-court persons (one entry per sampled frame)."""
83         return [
84             sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
court_polygon))
85             for _frame_idx, dets in sorted(per_frame.items())
86         ]
87
88
89     def _median(vals: Sequence[float]) -> float:
90         if not vals:
91             return 0.0
92         s = sorted(vals)
93         n = len(s)
94         return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
95
96
97     def players_in_court(counts: Sequence[int]) -> float:
98         """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
99         return _median(list(counts))
100
101
102     def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
103         """Fraction of sampled frames with >= `min_players` in court (track stability vs
flicker)."""
104         if not counts:
105             return 0.0
106         return sum(1 for c in counts if c >= min_players) / len(counts)
107
108
109     def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
110                           frame_width: float, frame_height: float) -> float:
111         """Mean per-track centre displacement between consecutive sampled frames, with x
112         normalised by frame width and y by frame height (per-axis; uniform-scale-invariant,
113         not aspect-ratio-isotropic). 0.0 if <2 frames or no shared tracks."""
114         if frame_width <= 0 or len(per_frame) < 2:
115             return 0.0
116         frames = sorted(per_frame.keys())
117         steps: List[float] = []
118         for a, b in zip(frames, frames[1:]):
119             prev = {tid: box for tid, box in per_frame[a]}
120             for tid, box in per_frame[b]:
121                 if tid in prev:
122                     cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
123                     cx1, cy1 = _center_norm(box, frame_width, frame_height)
124                     steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
125         if not steps:
126             return 0.0
127         return sum(steps) / len(steps)
128
129
130     def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
131                         frame_height: float, net: Optional[Net],
132                         court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
133         """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
134
135         A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
136         The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
0.0.

```

```

137     """
138     if net is None or not per_frame:
139         return 0.0
140     axis, pos = net
141     both = 0
142     for _frame_idx, dets in per_frame.items():
143         near, far = False, False
144         for _tid, box in dets:
145             if not person_in_court(box, frame_width, frame_height, court_polygon):
146                 continue
147             fx, fy = _foot_norm(box, frame_width, frame_height)
148             coord = fx if axis == "x" else fy
149             if coord < pos:
150                 near = True
151             else:
152                 far = True
153         if near and far:
154             both += 1
155     return both / len(per_frame)
156
157
158 def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
159                    pad_frac: float) -> bool:
160     """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
161     width/height (the 'adjacent / in-hand' tolerance)?"""
162     x1, y1, x2, y2 = box
163     px = pad_frac * frame_width
164     py = pad_frac * frame_height
165     return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
166
167
168 def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
169                               frame_width: float, frame_height: float,
170                               pad_frac: float = 0.03) -> float:
171     """Fraction of (visible shuttle frames that were also person-sampled) in which the
172     shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
173     ~0**
174     ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
175     """
176     if not per_frame or frame_width <= 0 or frame_height <= 0:
177         return 0.0
178     aligned = 0
179     collocated = 0
180     for p in shuttle_points or []:
181         if not getattr(p, "visible", False):
182             continue
183         dets = per_frame.get(int(p.frame))
184         if dets is None: # shuttle frame wasn't person-sampled ? can't judge
185             continue
186         aligned += 1
187         if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
188               for _tid, box in dets):
189             collocated += 1
190     return collocated / aligned if aligned else 0.0
191
192 # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
193 # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
194 PERSON_FEATURE_NAMES = [
195     "players_in_court", "two_sided_motion", "mean_player_motion",
196     "in_court_ratio", "shuttle_player_colocation",
197 ]
198

```

```

199
200     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
201                                 frame_width: float, frame_height: float,
202                                 court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
203                                 net: Optional[Net] = None,
204                                 min_players: int = 2) -> Dict[str, float]:
205     """Aggregate the per-window person features from per-frame detections + the in-
window
206     shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
207     counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
208     return {
209         "players_in_court": players_in_court(counts),
210         "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
211         "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
212         "in_court_ratio": in_court_ratio(counts, min_players),
213         "shuttle_player_colocation": shuttle_player_colocation(
214             per_frame, shuttle_points, frame_width, frame_height),
215     }
216

```

15. user (2026-06-13T19:55:44.597Z)

```

1     """Person-detection rally cue ? torchvision detector + ByteTrack + motion windowing.
2
3     Extracted verbatim from `yolo_hybrid` (fusion P1, `docs/MULTI_SIGNAL_FUSION_PLAN.md`):
4     the person cue is a swappable producer**, not logic baked into one segmenter. Today
5     `HybridYoloSegmenter` is the only consumer (it delegates Stage-1 here, byte-
identically);
6     next the fusion segmenter consumes the SAME producer over shuttle-seeded windows, and
7     later the learned classifier reads its persisted features. Keeping detection here (not
in
8     the segmenter) is what makes "add a cue = register a producer" true.
9
10    Behaviour is unchanged from the in-lined version ? only the home moved:
11    - `lazy_load_model`           ? load a permissively-licensed torchvision detector
12    - `make_tracker`             ? a fresh supervision ByteTrack per chunk
13    - `detect_and_track`         ? one frame ? confident persons with persistent IDs
14    - `extract_action_windows_from_chunk` ? a chunk ? high-motion candidate windows
15    with the motion stats (`peak/mean_velocity`, `active_frame_count`, `track_id_count`)
16    that feed the candidate `stats` JSON and the learned fusion features.
17
18    The model libs (torchvision, supervision) are imported lazily inside the methods, and
the
19    detector weights load only on first detection (`lazy_load_model`) ? so importing the
20    segmenter registry never loads a detector model, and tests can mock the seams. (torch +
cv2
21    are top-level imports here; CI's import-smoke stubs them, and they're present on the GPU
22    machine.) Licensing: torchvision detectors are BSD + COCO weights; ByteTrack is MIT
23    (ADR-005; ultralytics' AGPL YOLO was dropped).
24    """
25    import logging
26    from typing import Any, Dict, List, Optional, Tuple
27
28    import cv2
29    import torch
30
31    from backend.utils.geometry import get_velocity_normalised
32
33    logger = logging.getLogger(__name__)
34
35    # torchvision detection models are trained on COCO where the "person" category is
36    # class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?

```

```

37 # this off-by-one is the classic gotcha when swapping detectors. Keep it named.
38 _PERSON_CLASS_ID = 1
39
40 # Permissively-licensed torchvision detectors selectable via config
41 # `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
42 _DETECTOR_FACTORIES = {
43     "fasterrcnn_resnet50_fpn": "fasterrcnn_resnet50_fpn",
44     "fasterrcnn_mobilenet_v3_large_fpn": "fasterrcnn_mobilenet_v3_large_fpn",
45     "retinanet_resnet50_fpn": "retinanet_resnet50_fpn",
46 }
47 _DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"
48
49
50 class PersonDetector:
51     """Stage-1 person detection + tracking + motion-window extraction (a rally cue).
52
53     Owns the torchvision model + per-chunk ByteTrack tracker. Constructed with the run
54     ``config`` (reads ``indexing.{use_gpu, detector_model, detector_score_threshold,
55     min_action_window_duration}``); the model loads lazily on first use.
56     """
57
58     def __init__(self, config: Dict[str, Any]):
59         self.config = config
60         # Dynamic torchvision module (None until lazy_load_model); typed Any so the
61         # `self.model([tensor])` inference call type-checks without a quarantine.
62         self.model: Any = None
63         self.device: Optional[str] = None
64
65     def lazy_load_model(self) -> None:
66         if self.model is not None:
67             return
68
69         # Imported lazily so the module imports without torchvision present and so
70         # the test suite (which pre-sets self.model) never pulls in model weights.
71         from torchvision.models import detection as tv_detection
72
73         idx_cfg = self.config.get("indexing", {})
74         use_gpu = idx_cfg.get("use_gpu", True)
75         self.device = "cuda" if use_gpu and torch.cuda.is_available() else "cpu"
76         if use_gpu and not torch.cuda.is_available():
77             logger.warning("use_gpu is True but CUDA is not available. Falling back to
CPU.")
78
79         model_name = idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
80         if model_name not in _DETECTOR_FACTORIES:
81             logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
{_DEFAULT_DETECTOR}.")
82             model_name = _DEFAULT_DETECTOR
83
84         builder = getattr(tv_detection, _DETECTOR_FACTORIES[model_name])
85         logger.info(f"Loading torchvision detector '{model_name}' on {self.device}...")
86         # weights="DEFAULT" pulls the best available COCO-pretrained checkpoint.
87         self.model = builder(weights="DEFAULT")
88         self.model.eval()
89         self.model.to(self.device)
90
91     def make_tracker(self, fps: float) -> Any:
92         """Create a fresh ByteTrack tracker for one chunk (track IDs are chunk-local).
93
94         Imported lazily so tests can mock this method without supervision installed.
95         """
96         import supervision as sv
97         frame_rate = max(1, int(round(fps)))
98         return sv.ByteTrack(frame_rate=frame_rate)
99

```



```

100     def detect_and_track(self, frame: Any, tracker: Any,
101                           score_thresh: float) -> List[Tuple[int, Tuple[float, float,
float, float]]]:
102         """Detect persons in a frame and assign persistent track IDs.
103
104         Returns a list of (track_id, bbox) where bbox is (x1, y1, x2, y2) in pixels.
105         This replaces the old ultralytics ``model.track(...)`` call, which bundled
106         detection AND association: torchvision does the detection, supervision's
107         ByteTrack does the association.
108         """
109         import supervision as sv
110
111         # cv2 frames are BGR uint8 HxWxC; torchvision wants RGB float [0,1] CxHxW.
112         rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
113         tensor = torch.from_numpy(rgb).permute(2, 0,
1).float()).div(255.0).to(self.device)
114         with torch.no_grad():
115             outputs = self.model([tensor])
116
117         out = outputs[0]
118         boxes = out["boxes"].detach().cpu().numpy()
119         labels = out["labels"].detach().cpu().numpy()
120         scores = out["scores"].detach().cpu().numpy()
121
122         # Keep only confident person detections.
123         mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)
124         if not mask.any():
125             return []
126
127         # ByteTrack requires confidence to be populated (it filters on it internally).
128         dets = sv.Detections(xyxy=boxes[mask], confidence=scores[mask],
class_id=labels[mask])
129         tracked = tracker.update_with_detections(dets)
130
131         result: List[Tuple[int, Tuple[float, float, float, float]]] = []
132         for i in range(len(tracked)):
133             tid = tracked.tracker_id[i]
134             if tid is None:
135                 continue
136             x1, y1, x2, y2 = tracked.xyxy[i]
137             result.append((int(tid), (float(x1), float(y1), float(x2), float(y2))))
138         return result
139
140     def extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
141                                           velocity_thresh: float, merge_gap: float,
142                                           frame_skip: int = 3,
143                                           chunk_index: Optional[int] = None) ->
List[Dict[str, Any]]:
144         """
145         Runs person detection + tracking on a video chunk and extracts high-motion
146         action windows. Returns a list of dicts in the full video's timeframe, each
with:
147             start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
148             track_id_count, chunk_index.
149
150         Args:
151             frame_skip: Process every Nth frame (1 = every frame). Higher values
152                 dramatically reduce inference cost with minimal quality loss.
153             chunk_index: Index of the source chunk (recorded on each window for
traceability).
154         """
155         cap = cv2.VideoCapture(chunk_path)
156         if not cap.isOpened():
157             logger.error(f"Could not open {chunk_path}")
158             return []

```

```

159
160     fps = cap.get(cv2.CAP_PROP_FPS)
161     if fps <= 0:
162         fps = 30.0
163
164     frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
165     if frame_width <= 0:
166         frame_width = 1280.0 # Sensible fallback
167
168     # Effective sampling rate after frame-skipping
169     effective_fps = fps / max(frame_skip, 1)
170
171     # A fresh tracker per chunk ? track IDs only need to be consistent within a
172     # chunk (windows are computed per chunk, then merged across chunks by time).
173     tracker = self.make_tracker(effective_fps)
174     score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
175
176     # Per active frame we keep (frame_idx, peak_velocity, set_of_active_track_ids)
177     # so windows can be enriched with motion stats for logging + fine-tuning.
178     active_frames = []
179     frame_idx = 0
180     track_history: Dict[int, Tuple[float, float, float, float]] = {}
181
182     while True:
183         ret, frame = cap.read()
184         if not ret:
185             break
186
187         # Skip frames for performance ? at 30fps, every 3rd frame still
188         # gives ~10 samples/second, plenty for human-scale velocity.
189         if frame_idx % max(frame_skip, 1) != 0:
190             frame_idx += 1
191             continue
192
193         # Detect persons + assign persistent track IDs.
194         detections = self.detect_and_track(frame, tracker, score_thresh)
195
196         frame_peak_velocity = 0.0
197         frame_active_ids = set()
198         current_ids = set()
199
200         for track_id, bbox in detections:
201             current_ids.add(track_id)
202
203             if track_id in track_history:
204                 prev_bbox = track_history[track_id]
205                 velocity = get_velocity_normalised(prev_bbox, bbox, effective_fps,
frame_width)
206
207                 if velocity > velocity_thresh:
208                     frame_peak_velocity = max(frame_peak_velocity, velocity)
209                     frame_active_ids.add(track_id)
210
211                 track_history[track_id] = bbox
212
213             # Prune stale IDs that are no longer visible ? prevents false
214             # velocity spikes when the tracker reassigns an old ID.
215             track_history = {k: v for k, v in track_history.items() if k in current_ids}
216
217             if frame_active_ids:
218                 active_frames.append((frame_idx, frame_peak_velocity, frame_active_ids))
219
220             frame_idx += 1
221
222     cap.release()

```

```

222
223     # Group active frames into continuous windows
224     if not active_frames:
225         return []
226
227     max_gap_frames = int(merge_gap * fps)
228
229     def _finalize(group: List[Tuple[int, float, set]]) -> Dict[str, Any]:
230         velocities = [v for _, v, _ in group]
231         track_ids = set().union(*[ids for _, _, ids in group])
232         return {
233             "start": group[0][0] / fps + chunk_start,
234             "end": group[-1][0] / fps + chunk_start,
235             "active_frame_count": len(group),
236             "peak_velocity": max(velocities),
237             "mean_velocity": sum(velocities) / len(velocities),
238             "track_id_count": len(track_ids),
239             "chunk_index": chunk_index,
240         }
241
242     windows = []
243     group = [active_frames[0]]
244     for af in active_frames[1:]:
245         if af[0] - group[-1][0] <= max_gap_frames:
246             group.append(af)
247         else:
248             windows.append(_finalize(group))
249             group = [af]
250     windows.append(_finalize(group))
251
252     # Filter out extremely short action windows
253     min_window_duration = self.config.get("indexing",
254     {}).get("min_action_window_duration", 2.0)
255     windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
256
257     return windows
258
259     def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
260     frame_skip: int = 3) -> Dict[str, Any]:
261         """Per-frame person detections over ``[start_frame, end_frame]`` of the ORIGINAL
262         video.
263         Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
264         ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and
265         discards the
266         per-frame boxes), this returns the raw per-frame detections so the fusion
267         features can
268         align them with the shuttle trajectory. It reads the source by **absolute frame
269         index**
270         (seek + sequential read), NOT a re-extracted clip, so the returned frame indices
271         match
272         the WASB trajectory's ``.frame`` exactly ? that alignment is what
273         ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
274
275         Returns ``{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
276         "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-
277         seek
278         accuracy varies by codec (it lands near the requested keyframe then decodes
279         forward);
280         treat the indices as ±a few frames on real footage ? fine for window-level
281         features.
282         """
283         self.lazy_load_model()
284         cap = cv2.VideoCapture(video_path)
285         if not cap.isOpened():

```

```

278         logger.error(f"Could not open {video_path}")
279         return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
280
281     fps = cap.get(cv2.CAP_PROP_FPS)
282     if fps <= 0:
283         fps = 30.0
284     frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
285     frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
286     score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
287     step = max(frame_skip, 1)
288     tracker = self.make_tracker(fps / step)
289
290     start_frame = max(0, int(start_frame))
291     frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
292     cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
293     frame_idx = start_frame
294     while frame_idx <= end_frame:
295         ret, frame = cap.read()
296         if not ret:
297             break
298         if (frame_idx - start_frame) % step == 0:
299             frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
300             frame_idx += 1
301     cap.release()
302     return {"frames": frames, "fps": fps,
303           "frame_width": float(frame_width), "frame_height": float(frame_height)}
304

```

16. user (2026-06-13T19:55:44.776Z)

```

1     """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4     inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6         1. Translate the Windows video path to its WSL-visible form (and optionally
7           stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8         2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9         3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10        4. Convert that trajectory into high-motion "action windows" ? the same
11           candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12           pipeline (AI handoff, DB population) is unchanged.
13
14     Nothing here imports TensorFlow; all model code stays in WSL.
15     """
16
17     import os
18     import csv
19     import shlex
20     import logging
21     import subprocess
22     from dataclasses import dataclass
23     from typing import List, Dict, Any, Optional, Tuple
24
25     from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27     logger = logging.getLogger(__name__)
28
29
30     @dataclass
31     class TrackNetConfig:
32         """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""

```

```

33     distro: str = "Ubuntu"
34     repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
35     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36     conda_env: str = "TrackNetV4"
37     weights_path: str = ""                          # WSL path to the .h5/.keras weights
(REQUIRED)
38     queue_length: int = 5
39     stage_in_wsl: bool = True                       # copy video into WSL fs first
(perf)
40     wsl_stage_dir: str = "~/clips"                  # where staged videos/outputs live
in WSL
41     timeout_sec: int = 1800                         # 30 min; kills a hung call (0 = no
timeout)
42     keep_frames: bool = False                       # retain staged video after success
(debug only)
43     min_free_gb: float = 0.0                        # warn if WSL free space below this
(0 = off)
44
45     @classmethod
46     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
47         tn = (idx_cfg or {}).get("tracknet", {}) or {}
48         return cls(
49             distro=tn.get("wsl_distro", "Ubuntu"),
50             repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
51             conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52             conda_env=tn.get("conda_env", "TrackNetV4"),
53             weights_path=tn.get("weights_path", ""),
54             queue_length=int(tn.get("queue_length", 5)),
55             stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
56             wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
57             timeout_sec=int(tn.get("timeout_sec", 1800)),
58             keep_frames=bool(tn.get("keep_frames", False)),
59             min_free_gb=float(tn.get("min_free_gb", 0.0)),
60         )
61
62
63     def to_wsl_mnt_path(win_path: str) -> str:
64         """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
65
66         Already-POSIX paths (starting with / or ~) are returned unchanged so the
67         same helper is safe to call on either kind of path.
68         """
69         p = str(win_path)
70         if p.startswith("/") or p.startswith("~"):
71             return p
72         p = p.replace("\\", "/")
73         if len(p) >= 2 and p[1] == ":":
74             drive = p[0].lower()
75             return f"/mnt/{drive}{p[2:]}"
76         return p
77
78
79     @dataclass
80     class TrajectoryPoint:
81         frame: int
82         visible: bool
83         x: float
84         y: float
85
86
87     class TrackNetRunner(WSLCommandMixin, DetectorRunner):
88         """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
89
90         Implements the common DetectorRunner contract so a future Linux-native or
91         alternative trajectory runner can be substituted without touching the hybrids.

```

```

92     """
93
94     def __init__(self, cfg: TrackNetConfig):
95         self.cfg = cfg
96
97     def healthcheck(self) -> Tuple[bool, str]:
98         """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
99
100         Returns (ok, message). Cheap to call before a long run.
101         """
102         cmd = (
103             f"conda activate {self.cfg.conda_env} && "
104             f"python -c \"import tensorflow as tf; "
105             f"print('TF', tf.__version__, 'GPUs', "
106             f"len(tf.config.list_physical_devices('GPU')))\n"
107         )
108         try:
109             res = self._wsl_bash(cmd)
110             except FileNotFoundError:
111                 return False, "`wsl` not found ? is this running on Windows with WSL2
112                 installed?"
113             except subprocess.TimeoutExpired:
114                 return False, "WSL healthcheck timed out."
115             out = (res.stdout or "").strip()
116             if res.returncode != 0:
117                 return False, f"WSL env check failed: {(res.stderr or out).strip()}"
118             return True, out
119
120     # ----- inference
121
122     def run_predict(self, video_win_path: str, output_win_dir: str,
123                    video_id: Optional[str] = None) -> Optional[str]:
124         """Run predict.py on a video. Returns the Windows path to the produced CSV, or
125         None.
126
127         ``output_win_dir`` is a Windows directory (under the repo's output/) that
128         WSL writes into via its /mnt view, so the Windows process can read the CSV
129         back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
130         therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
131         a filename cannot collide on the output CSV.
132         """
133         if not self.cfg.weights_path:
134             logger.error(
135                 "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
136                 "in config.json to the WSL path of your trained TrackNetV4 weights."
137             )
138             return None
139
140         # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
141         # relative paths through unchanged, so WSL would resolve them against the
142         # predict.py cwd instead of the repo (same bug class as wasb_runner).
143         output_win_dir = os.path.abspath(output_win_dir)
144         os.makedirs(output_win_dir, exist_ok=True)
145         # Normalize separators so basename/splitext work when tests (or collector)
146         # pass Windows-style paths on a Linux host.
147         video_win_path = str(video_win_path).replace("\\", "/")
148         base = os.path.basename(video_win_path)
149         stem, ext = os.path.splitext(base)
150
151         # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
152         if ext.lower() not in (".mp4", ".avi"):
153             logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
154             return None
155
156         wsl_out = to_wsl_mnt_path(output_win_dir)

```

```

154         # Namespace by video_id so two same-filename videos don't collide on the CSV.
155         # predict.py derives its output name from the (staged) video stem, so staging
156         # under the namespaced name propagates into "<key>_predict.csv".
157         key = f"{stem}__{video_id[:12]}" if video_id else stem
158         expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
159
160         # Resolve the video path WSL will read.
161         if self.cfg.stage_in_wsl:
162             staged = self._stage_video(video_win_path, f"{key}{ext}")
163             if staged is None:
164                 return None
165             wsl_video = staged
166         else:
167             # No staging: predict.py reads /mnt directly and writes
168             # We cannot rename its output, so fall back to the un-namespaced name.
169             wsl_video = to_wsl_mnt_path(video_win_path)
170             expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
171
172         if self.cfg.min_free_gb > 0:
173             free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
174             if free is not None and free < self.cfg.min_free_gb:
175                 logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
176                               "consider running tools/cleanup_caches.py.",
177                               free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
178
179         cmd = (
180             f"conda activate {self.cfg.conda_env} && "
181             f"cd {self.cfg.repo_dir}/src && "
182             f"python predict.py "
183             f"--video_path {wsl_tilde_quote(wsl_video)} "
184             f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
185             f"--output_dir {wsl_tilde_quote(wsl_out)} "
186             f"--queue_length {self.cfg.queue_length}"
187         )
188         logger.info("Running TrackNet inference on %s ...", base)
189         try:
190             res = self._wsl_bash(cmd)
191         except subprocess.TimeoutExpired:
192             logger.error("TrackNet inference timed out after %ss.",
193                         self.cfg.timeout_sec)
194             return None
195
196         if res.returncode != 0:
197             logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
198                     res.stdout)[-2000:])
199             return None
200
201         if not os.path.exists(expected_csv_win):
202             logger.error("TrackNet finished but expected CSV not found at %s",
203                         expected_csv_win)
204             return None
205         logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
206
207         # Success ? the CSV is the durable output; drop the staged video copy (a pure,
208         # regenerable intermediate). On earlier failure we return above without
209         # cleaning.
210         if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
211             logger.info("Cache hygiene: removing staged video for %s "
212                         "(set indexing.tracknet.keep_frames=true to retain).", key)
213             self._wsl_rm_rf([wsl_video])
214         return expected_csv_win
215
216 def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
217     """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).

```

```

214
215         Returns the WSL path to the staged copy, or None on failure.
216         """
217         src_mnt = to_wsl_mnt_path(video_win_path)
218         dest = f"{self.cfg.wsl_stage_dir}/{base}"
219         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
220         try:
221             res = self._wsl_bash(cmd)
222         except subprocess.TimeoutExpired:
223             logger.error("Staging video into WSL timed out.")
224             return None
225         if res.returncode != 0 or "OK" not in (res.stdout or ""):
226             logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
227             return None
228         return dest
229
230     # ----- CSV -> windows
231
232     @staticmethod
233     def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
234         """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
235         points: List[TrajectoryPoint] = []
236         with open(csv_win_path, newline="") as f:
237             reader = csv.DictReader(f)
238             for row in reader:
239                 try:
240                     points.append(TrajectoryPoint(
241                         frame=int(row["Frame"]),
242                         visible=int(row["Visibility"]) == 1,
243                         x=float(row["X"]),
244                         y=float(row["Y"]),
245                     ))
246                 except (KeyError, ValueError):
247                     continue
248             points.sort(key=lambda p: p.frame)
249         return points
250
251     @staticmethod
252     def trajectory_to_action_windows(
253         points: List[TrajectoryPoint],
254         fps: float,
255         frame_width: float,
256         chunk_start: float = 0.0,
257         velocity_thresh: float = 0.02,
258         merge_gap: float = 2.0,
259         min_window_duration: float = 2.0,
260         chunk_index: Optional[int] = None,
261     ) -> List[Dict[str, Any]]:
262         """Convert a shuttle trajectory into candidate rally windows.
263
264         A frame is "active" when the shuttle is visible AND its inter-frame
265         displacement (normalised by frame width, per second) exceeds
266         ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
267         ``merge_gap`` seconds of each other are grouped into one window; short
268         windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
269         shape feeding the AI-handoff phase is identical.
270         """
271         if fps <= 0:
272             fps = 30.0
273         if frame_width <= 0:
274             frame_width = 1280.0
275
276         # Compute per-frame normalised velocity from the last *visible* point.

```



```

277         active = [] # (frame_idx, velocity)
278         prev: Optional[TrajectoryPoint] = None
279         for p in points:
280             if not p.visible:
281                 prev = None # break the trajectory; absent shuttle = not in flight
282                 continue
283             if prev is not None:
284                 dframes = p.frame - prev.frame
285                 if dframes > 0:
286                     dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
287                     velocity = (dist / frame_width) * (fps / dframes)
288                     if velocity > velocity_thresh:
289                         active.append((p.frame, velocity))
290             prev = p
291
292         if not active:
293             return []
294
295         max_gap_frames = int(merge_gap * fps)
296
297         def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
298             vels = [v for _, v in group]
299             return {
300                 "start": group[0][0] / fps + chunk_start,
301                 "end": group[-1][0] / fps + chunk_start,
302                 "active_frame_count": len(group),
303                 "peak_velocity": max(vels),
304                 "mean_velocity": sum(vels) / len(vels),
305                 "track_id_count": 1, # single shuttle; kept for window-shape parity
306                 "chunk_index": chunk_index,
307             }
308
309         windows = []
310         group = [active[0]]
311         for af in active[1:]:
312             if af[0] - group[-1][0] <= max_gap_frames:
313                 group.append(af)
314             else:
315                 windows.append(_finalize(group))
316                 group = [af]
317         windows.append(_finalize(group))
318
319         return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
320

```

17. assistant (2026-06-13T19:55:46.809Z)

Now let me examine the `distill_local` module to understand `build_candidates`, and the training/experiment harness:

18. user (2026-06-13T19:55:47.073Z)

```

tests\test_distill_local.py
tests\__pycache__\test_distill_local.cpython-313-pytest-9.0.3.pyc
tests\__pycache__\test_distill_local.cpython-313.pyc
backend\eval\distill_local.py
backend\eval\__pycache__\distill_local.cpython-313.pyc

```

19. user (2026-06-13T19:55:47.353Z)

```
backend\eval\experiment.py
```

20. user (2026-06-13T19:55:47.481Z)

No files found

21. user (2026-06-13T19:55:49.156Z)

```
1      """Distill Gemini (teacher) into a LEARNED local rally classifier (student).
2
3      Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
4      further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
5      using a few resolution/fps-INVARIANT local features, so the model transfers across
6      videos and runs 100% local at inference (no Gemini, privacy lever intact).
7
8      Flow (all local at runtime):
9          WASB trajectory -> recall-oriented candidate windows (propose many; classifier
10             filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
11             active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
12             -> rally windows.
13
14      Training labels come from Gemini full-context labels via the SAME IoU matching the
15      evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
16      other videos' candidates, predict the held-out one, score vs its Gemini labels, and
17      compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
18      rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.
19
20      Run:
21          python -m backend.eval.distill_local --manifest videos.json --model-out
output/local_rally_model.json
22      """
23      from __future__ import annotations
24
25      import json
26      import argparse
27      from typing import Dict, List, Optional, Tuple
28
29
30      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
31      from backend.pipeline.detectors import rally_gate
32      from backend.eval.calibrate_wasb import load_golden_gts
33      from backend.eval.training import label_candidates
34      from backend.eval.classifier import LogisticRegression
35      from backend.eval.metrics import score
36      from backend.eval.gemini_refine import merge_overlaps
37
38      Interval = Tuple[float, float]
39
40      FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
"net_crossings"]
41      # Recall-oriented proposal: deliberately permissive so real rallies are among the
42      # candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur)
43      PROPOSAL = (0.005, 1.0, 0.5)
44      BASELINE_LOCAL = (0.02, 2.0, 2.0) # today's production-ish windowing, no learning
45
46
47      def build_candidates(trajectory_csv: str, fps: float, frame_width: float,
48                          proposal: Tuple[float, float, float] = PROPOSAL) ->
Tuple[List[Interval], List[List[float]]]:
49          """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature
rows)."""
50          points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
51          vt, mg, mwd = proposal
52          wins = TrackNetRunner.trajectory_to_action_windows(
53              points, fps=fps, frame_width=frame_width,
54              velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
55          intervals = [(w["start"], w["end"]) for w in wins]
56          gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0) # for net-
crossings
```

```

57     X: List[List[float]] = []
58     for w, g in zip(wins, gated):
59         dur = max(1e-6, w["end"] - w["start"])
60         win_frames = max(1.0, dur * fps)
61         X.append([
62             dur,                                     # rally length (sec) ?
63             float(w["peak_velocity"]),               # already normalized by
64             float(w["mean_velocity"]),
65             float(w["active_frame_count"]) / win_frames, # active-shuttle ratio ?
66             float(g.crossings),                       # net crossings (count) ?
67             the rally signal
68         ])
69     return intervals, X
70
71     def baseline_windows(trajecory_csv: str, fps: float, frame_width: float) ->
72     List[Interval]:
73         points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
74         vt, mg, mwd = BASELINE_LOCAL
75         wins = TrackNetRunner.trajectory_to_action_windows(
76             points, fps=fps, frame_width=frame_width,
77             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
78         return [(w["start"], w["end"]) for w in wins]
79
80     def load_video(spec: Dict, iou: float = 0.5) -> Dict:
81         fps, fw = float(spec["fps"]), float(spec["frame_width"])
82         intervals, X = build_candidates(spec["trajectory"], fps, fw)
83         gts = load_golden_gts(spec["labels"])
84         y = label_candidates(intervals, gts, iou)
85         return {"name": spec.get("name", spec["trajectory"]), "intervals": intervals, "X":
86             X,
87             "y": y, "gts": gts, "baseline": baseline_windows(spec["trajectory"], fps,
88             fw)}
89
90     def predict_rallies(model: LogisticRegression, X: List[List[float]], intervals:
91     List[Interval],
92     threshold: float = 0.5) -> List[Interval]:
93         if not intervals:
94             return []
95         proba = model.predict_proba(X)
96         pos = [iv for iv, p in zip(intervals, proba) if p >= threshold]
97         return merge_overlaps(pos, gap_tol=1.0)
98
99     def run(specs: List[Dict], iou: float = 0.5, threshold: float = 0.5,
100     model_out: Optional[str] = None) -> dict:
101         videos = [load_video(s, iou) for s in specs]
102         print(f"=== Distilled local rally classifier vs Gemini silver-GT "
103             f"({len(videos)} videos, IoU>={iou}, prob>={threshold}) ===")
104         for v in videos:
105             ceil = score(v["intervals"], v["gts"], iou).recall
106             print(f"[{v['name']}] {len(v['intervals'])} candidates ({sum(v['y'])} pos) | "
107                 f"proposal recall ceiling {ceil:.3f} | {len(v['gts'])} Gemini rallies")
108         result: dict = {"videos": [v["name"] for v in videos]}
109         if len(videos) >= 2:
110             print("\n--- HONEST leave-one-video-out: learned vs threshold baseline (held-
111             out) ---")
112             clf, base = [], []
113             for i, held in enumerate(videos):

```

```

113         others = [v for j, v in enumerate(videos) if j != i]
114         X = [row for v in others for row in v["X"]]
115         y = [t for v in others for t in v["y"]]
116         model = LogisticRegression().fit(X, y, FEATURE_NAMES)
117         preds = predict_rallies(model, held["X"], held["intervals"], threshold)
118         sc = score(preds, held["gts"], iou)
119         bsc = score(held["baseline"], held["gts"], iou)
120         clf.append(sc)
121         base.append(bsc)
122         print(f" held-out {held['name']}: LEARNED P={sc.precision:.3f}
R={sc.recall:.3f} F1={sc.f1:.3f}"
123               f" | baseline P={bsc.precision:.3f} R={bsc.recall:.3f}
F1={bsc.f1:.3f}")
124         def mean(xs, a):
125             return sum(getattr(s, a) for s in xs) / len(xs)
126             print(f" MEAN held-out: LEARNED P={mean(clf, 'precision'):.3f}
R={mean(clf, 'recall'):.3f} "
127                   f"F1={mean(clf, 'f1'):.3f} | baseline P={mean(base, 'precision'):.3f} "
128                   f"R={mean(base, 'recall'):.3f} F1={mean(base, 'f1'):.3f}")
129             result["mean_learned_f1"] = mean(clf, "f1")
130             result["mean_baseline_f1"] = mean(base, "f1")
131
132         # Final model on ALL videos (for production / running on a fresh held-out video).
133         Xall = [row for v in videos for row in v["X"]]
134         yall = [t for v in videos for t in v["y"]]
135         final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
136         assert final.w is not None # set by fit()
137         weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
138         print(f"\nfinal model standardized weights (sign/size = influence): {weights}")
139         if model_out:
140             final.save(model_out)
141             print(f"final model (all videos) -> {model_out}")
142             result["model_out"] = model_out
143         return result
144
145
146     def main() -> None:
147         ap = argparse.ArgumentParser(description="Distill Gemini labels into a local rally
classifier.")
148         ap.add_argument("--manifest", required=True,
149                         help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]
(labels = Gemini CSV)")
150         ap.add_argument("--iou", type=float, default=0.5)
151         ap.add_argument("--threshold", type=float, default=0.5, help="classifier probability
cutoff")
152         ap.add_argument("--model-out", default="output/local_rally_model.json")
153         args = ap.parse_args()
154         with open(args.manifest, encoding="utf-8") as f:
155             specs = json.load(f)
156         run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)
157
158
159     if __name__ == "__main__":
160         main()
161

```

22. user (2026-06-13T19:55:49.414Z)

```

1     """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3     This is the thin **variant/experiment layer** that `docs/EXPERIMENT_HARNESS.md`
4     identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5     no-regression gate, the pinnable classifier artifact, the persisted-candidate
6     substrate) already exists ? this module composes it. **No new scoring math lives
7     here**; everything defers to:

```

```

8
9     - `backend.eval.metrics`           ? `score`, `match_intervals`, `interval_iou`
10    - `backend.eval.training`          ? `label_candidates` (y by the same IoU matcher)
11    - `backend.eval.classifier`        ? `LogisticRegression` (the pinnable JSON model)
12    - `backend.eval.gemini_refine`     ? `merge_overlaps` (the blessed window merge)
13    - `backend.eval.regression`        ? `gt_hash` (label-set fingerprint)
14    - `backend.eval.calibration`       ? `pct_improvement`
15
16    The thesis (`EXPERIMENT_HARNESS.md` §2): separate the expensive, risky DETECTION
17    (per-frame signals ? run once on the GPU/WSL box, persisted into
18    `candidate_segments.stats`) from the cheap, swappable DECISION (given those
19    signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20    given persisted candidate features it produces `List[Interval]` deterministically,
21    with no GPU and zero production risk. So most experiments are pure offline
22    re-scoring of stored data and never touch the served pipeline.
23
24    Three modes (`EXPERIMENT_HARNESS.md` §5):
25    - `compare()` ? labeled A/B vs golden labels: per-video + LOVO-honest
26      aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27      held-out loop, but variant-parameterised.
28    - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29      variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30      human spot-checks (and what two highlight reels would show side by side).
31    - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32      (exit 0/1/3); rollback = flip the variant/config, never `git revert`.
33
34    Pure + offline + unit-tested. The only DB-touching helper is
35    `load_video_candidates`, which reads persisted candidates via
36    `Database.query_candidate_segments`.
37    """
38    from __future__ import annotations
39
40    import json
41    import logging
42    import os
43    from dataclasses import dataclass, field
44    from datetime import datetime, timezone
45    from hashlib import sha1
46    from typing import Any, Dict, List, Optional, Sequence
47
48    from backend.eval.calibration import pct_improvement
49    from backend.eval.classifier import LogisticRegression
50    from backend.eval.gemini_refine import merge_overlaps
51    from backend.eval.metrics import Interval, match_intervals, score
52    from backend.eval.regression import gt_hash
53    from backend.eval.training import label_candidates
54
55    logger = logging.getLogger(__name__)
56
57    # Where a comparison run appends one reproducible record. Tracked location (NOT under
58    # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
59    # regression.DEFAULT_BASELINE_PATH.
60    DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
61    _METRICS = ("precision", "recall", "f1", "mean_iou")
62
63
64    class ExperimentError(ValueError):
65        """A variant cannot be evaluated as configured (e.g. a learned variant asked to
66        score an unlabeled video with no pinned model, or a gate referencing an absent
67        feature)."""
68
69
70    # ----- Variant

```

```

73 @dataclass
74 class Variant:
75     """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
76
77     A variant turns one video's persisted candidate windows + features into rally
78     intervals. Every knob defaults to the control behaviour (no gate, no learned
79     filter), so a variant that merely carries a new, disabled cue is byte-identical
80     to control ? the core no-regression guarantee.
81
82     Fields:
83     - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
84       for the leaderboard and for selecting the served path (the existing
85       ``segmenter_registry`` + ``IndexingConfig`` do the actual selection in production).
86       New cues are recorded here default-OFF.
87     - ``min_crossings`` ? decision-gate Gate A: keep only windows whose persisted
88       ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
89       ``rally_gate.py`` expressed as a re-scoring knob (kills service-feed / held-
shuttle
90       windows ? see ``MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
91     - ``min_players`` ? decision-gate Gate B (the future person cue): keep windows
92       whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
93       cue persists that feature).
94     - ``classifier_model`` ? path to a frozen ``LogisticRegression`` JSON. When set,
95       ``compare()`` scores videos with the SHIPPED model as-is (no per-fold training);
96       required for ``compare_unlabeled`` on a learned variant.
97     - ``feature_names`` ? the persisted features the learned filter consumes. When set
98       WITHOUT ``classifier_model``, ``compare()`` trains a fresh model per LOVO fold
99       (honest) ? the recipe, not a pinned artifact.
100     - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
101       overlap-merge gap (seconds), the same ``gemini_refine.merge_overlaps`` default.
102     """
103
104     name: str
105     segmenter: str = "wasb_hybrid"
106     config_overrides: Dict[str, Any] = field(default_factory=dict)
107     cue_flags: Dict[str, bool] = field(default_factory=dict)
108     min_crossings: int = 0
109     min_players: int = 0
110     classifier_model: Optional[str] = None
111     feature_names: Optional[List[str]] = None
112     threshold: float = 0.5
113     merge_gap: float = 1.0
114
115     def is_learned(self) -> bool:
116         """True if this variant applies a learned classifier (pinned model or
recipe)."""
117         return self.classifier_model is not None or bool(self.feature_names)
118
119     def to_dict(self) -> dict:
120         return {
121             "name": self.name,
122             "segmenter": self.segmenter,
123             "config_overrides": self.config_overrides,
124             "cue_flags": self.cue_flags,
125             "min_crossings": self.min_crossings,
126             "min_players": self.min_players,
127             "classifier_model": self.classifier_model,
128             "feature_names": self.feature_names,
129             "threshold": self.threshold,
130             "merge_gap": self.merge_gap,
131         }
132
133     @classmethod
134     def from_dict(cls, d: dict) -> "Variant":
135         known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]

```

```

136         return cls(**{k: v for k, v in d.items() if k in known})
137
138     def spec_hash(self) -> str:
139         """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
140         and makes a result reproducible. The pinned **control** variant always hashes
141         the
142         same, so a regression is always traceable to a recipe change, never to drift."""
143         blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
144         return sha1(blob.encode()).hexdigest()[:12]
145
146     #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
147     #: filter. Always the comparison reference; "rollback" = make this the served variant.
148     CONTROL = Variant(name="control")
149
150
151     # ----- persisted candidates
152
153
154     @dataclass
155     class VideoCandidates:
156         """One video's persisted detection, ready for offline re-scoring.
157
158         ``intervals`` are the candidate windows; ``features`` is column-major
159         (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
160         ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
161         with ``load_video_candidates``, or directly in tests.
162         """
163
164         name: str
165         intervals: List[Interval]
166         features: Dict[str, List[float]] = field(default_factory=dict)
167         gts: Optional[List[Interval]] = None
168
169
170     def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
171         """Persisted feature column, defaulting missing/short columns to 0.0 (matches
172         ``training.extract_yolo_features``, which lets a sparse/old row still vectorise)."""
173         col = vc.features.get(name)
174         n = len(vc.intervals)
175         if col is None:
176             logger.warning("variant references feature %r absent from %s ? treating as 0.0",
177                             name, vc.name)
178             return [0.0] * n
179         if len(col) != n:
180             raise ExperimentError(
181                 f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
182         return [float(x) for x in col]
183
184
185     def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
186     List[List[float]]:
187         cols = [_feature_column(vc, name) for name in feature_names]
188         return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
189
190
191     def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
192         """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
193         players)."""
194         n = len(vc.intervals)
195         mask = [True] * n
196         if variant.min_crossings > 0:
197             col = _feature_column(vc, "net_crossings")
198             mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
199         if variant.min_players > 0:

```

```

198         col = _feature_column(vc, "players_in_court")
199         mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
200     return mask
201
202
203     def predict(variant: Variant, vc: VideoCandidates,
204                 model: Optional[LogisticRegression] = None) -> List[Interval]:
205         """Variant prediction: gate by persisted features, optionally filter by a learned
206         model, then merge. Pure and deterministic.
207
208         ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
209         fold; if omitted and the variant pins ``classifier_model``, that frozen model is
210         loaded. A learned variant with neither a supplied nor a pinned model raises ? there
211         is nothing to score with.
212         """
213         mask = _gate_mask(variant, vc)
214         if variant.feature_names:
215             if model is None:
216                 if variant.classifier_model is None:
217                     raise ExperimentError(
218                         f"variant {variant.name!r} is learned but has no model to predict "
219                         f"with (pass a fitted model or set classifier_model)")
220                 model = LogisticRegression.load(variant.classifier_model)
221                 proba = model.predict_proba(_feature_matrix(vc, variant.feature_names))
222                 mask = [m and (float(proba[i]) >= variant.threshold) for i, m in
223                         enumerate(mask)]
224                 kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
225                 return merge_overlaps(kept, gap_tol=variant.merge_gap)
226
227     # ----- variant scoring
228
229
230     def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
231     LogisticRegression:
232         """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
233         the
234         evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`. """
235         X: List[List[float]] = []
236         y: List[int] = []
237         for vc in videos:
238             if vc.gts is None:
239                 raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
240             X.extend(_feature_matrix(vc, variant.feature_names or []))
241             y.extend(label_candidates(vc.intervals, vc.gts, iou))
242         if not X:
243             raise ExperimentError("cannot train a learned variant on zero candidates")
244         return LogisticRegression().fit(X, y, variant.feature_names)
245
246     def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
247                         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
248         """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
249         mean aggregate.
250
251         Prediction policy:
252         - **static variant** (no learned filter): predict each video directly ? no
253         training,
254         so no leakage; per-video scoring is already honest.
255         - **learned, pinned model** (``classifier_model`` set): score every video with the
256         SHIPPED model. ? optimistic if those videos were in its training set ? use this
257         to
258         report "how the shipped artifact does here", not for the promotion decision.
259         - **learned recipe** (``feature_names``, no pinned model) + ``honest=True``: LOVO
260         ?

```



```

257         train on the OTHER videos, predict the held-out one. The number to trust.
258     - learned recipe + ``honest=False``: train on all, predict each (overfit; sanity
only).
259     """
260     for vc in videos:
261         if vc.gts is None:
262             raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")
263
264     per_video: List[Dict[str, Any]] = []
265     predictions: Dict[str, List[Interval]] = {}
266
267     recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
268     pinned_model = (LogisticRegression.load(variant.classifier_model)
269                     if variant.classifier_model is not None else None)
270     all_model = None
271     if recipe_lovo and not honest:
272         all_model = _train(variant, videos, iou)
273
274     for i, vc in enumerate(videos):
275         if recipe_lovo and honest:
276             others = [v for j, v in enumerate(videos) if j != i]
277             if not others:
278                 raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
279             model: Optional[LogisticRegression] = _train(variant, others, iou)
280             elif recipe_lovo: # honest=False ? one model over all
videos
281                 model = all_model
282             else: # static variant or pinned model
283                 model = pinned_model
284             preds = predict(variant, vc, model=model)
285             assert vc.gts is not None # guarded at function top
286             sc = score(preds, vc.gts, iou)
287             per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
288             predictions[vc.name] = preds
289
290     aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
291                  for m in _METRICS}
292     return {
293         "variant": variant.name,
294         "spec_hash": variant.spec_hash(),
295         "is_learned": variant.is_learned(),
296         "honest_lovo": recipe_lovo and honest,
297         "per_video": per_video,
298         "aggregate": aggregate,
299         "predictions": predictions,
300     }

```

23. user (2026-06-13T19:55:49.491Z)

```

training\README.md
training\requirements-train.txt
training\gen0\__init__.py
training\__init__.py
training\__pycache__\__init__.cpython-313.pyc
training\gen0\__pycache__\__init__.cpython-313.pyc
training\gen0\harness.py
training\gen0\__pycache__\harness.cpython-313.pyc

```

24. assistant (2026-06-13T19:55:51.229Z)

Now let me check the training module for label_candidates:

25. user (2026-06-13T19:55:51.919Z)

```
/usr/bin/bash: line 1: $null: ambiguous redirect
```

26. user (2026-06-13T19:55:52.223Z)

Found 3 files

```
backend\eval\experiment.py
backend\eval\distill_local.py
backend\eval\training.py
```

27. user (2026-06-13T19:55:53.546Z)

```
1      """Build a training set for the offline, API-free learned rally segmenter.
2
3      Phase 4 of the collector->indexer bridge (see docs/ROADMAP.md, ADR-004). The idea:
4      a detector (YOLO player-motion today, optionally WASB shuttle-motion later)
5      produces candidate rally windows with features; the collector's ground-truth
6      rally intervals tell us which windows are real. A lightweight classifier learns
7      that mapping, replacing the paid Gemini refinement with a local model.
8
9      This module is the *data layer* ? pure and unit-tested:
10
11      - `label_candidates` ? generic: label windows positive/negative by whether they
12        match a ground-truth rally (same IoU matching the evaluator uses, so training
13        labels are consistent with how we score).
14      - `extract_yolo_features` ? YOLO-specific feature vector from a candidate row's
15        persisted `stats`. Pluggable: a WASB substrate supplies its own feature_fn.
16      - `build_training_set` ? glue: rows + GT -> (X, y, intervals).
17
18      No model training or I/O here; that lives in the trainer/segmenter once the
19      substrate is chosen from the Phase 3 candidate-recall result.
20      """
21
22      from typing import Callable, Dict, List, Tuple
23
24      from backend.eval.metrics import Interval, match_intervals
25
26      # YOLO Stage-1 persists these per candidate window in `candidate_segments.stats`
27      # (see HybridYoloSegmenter). Duration is derived from the window bounds.
28      YOLO_FEATURE_NAMES = [
29          "duration", "peak_velocity", "mean_velocity", "active_frame_count",
30          "track_id_count",
31      ]
32
33      def extract_yolo_features(row: Dict) -> List[float]:
34          """Feature vector for one YOLO candidate row (dict from query_candidate_segments).
35
36          `stats` is already JSON-deserialized by the DB layer. Missing fields default
37          to 0.0 so a sparse/old row still yields a well-formed vector.
38          """
39          stats = row.get("stats") or {}
40          duration = float(row["end_time"]) - float(row["start_time"])
41          return [
42              duration,
43              float(stats.get("peak_velocity", 0.0)),
44              float(stats.get("mean_velocity", 0.0)),
45              float(stats.get("active_frame_count", 0)),
46              float(stats.get("track_id_count", 0)),
47          ]
48
49
50      def label_candidates(candidate_intervals: List[Interval], gt_intervals: List[Interval],
51                          iou_threshold: float = 0.5) -> List[int]:
```

```

52     """Label each candidate window 1 (real rally) or 0 (spurious).
53
54     A window is positive iff it is matched one-to-one to a ground-truth rally at
55     `iou_threshold` ? the *same* matching the evaluator uses for true positives,
56     so the classifier is trained on exactly what we score. Substrate-agnostic:
57     works for YOLO or WASB-derived windows alike.
58     """
59     mr = match_intervals(candidate_intervals, gt_intervals, iou_threshold)
60     positive = {m.pred_index for m in mr.matches}
61     return [1 if i in positive else 0 for i in range(len(candidate_intervals))]
62
63
64     def build_training_set(
65         candidate_rows: List[Dict],
66         gt_intervals: List[Interval],
67         iou_threshold: float = 0.5,
68         feature_fn: Callable[[Dict], List[float]] = extract_yolo_features,
69     ) -> Tuple[List[List[float]], List[int], List[Interval]]:
70         """Turn one video's candidate rows + ground truth into (X, y, intervals).
71
72         `feature_fn` is pluggable so a WASB substrate can supply shuttle-motion
73         features without changing the labeling logic. Returns the feature matrix `X`,
74         binary labels `y`, and the candidate intervals (handy for downstream merging).
75         """
76         intervals: List[Interval] = [
77             (float(r["start_time"]), float(r["end_time"])) for r in candidate_rows
78         ]
79         y = label_candidates(intervals, gt_intervals, iou_threshold)
80         X = [feature_fn(r) for r in candidate_rows]
81         return X, y, intervals
82

```

28. assistant (2026-06-13T19:55:55.170Z)

Now let me check if there are any existing manifests or output structures:

29. user (2026-06-13T19:55:56.046Z)

No files found

30. user (2026-06-13T19:55:56.509Z)

```

total 7344486
drwxr-xr-x 1 avidu 197609      0 Jun 14 01:12 .
drwxr-xr-x 1 avidu 197609      0 Jun 14 01:25 ..
-rw-r--r-- 1 avidu 197609 2352117156 Jun 12 22:29 GX010125_1080p.mp4
-rw-r--r-- 1 avidu 197609 1207870728 Jun 13 15:03 GX010125_highlights.mp4
-rw-r--r-- 1 avidu 197609 2083468872 Jun 13 20:19 GX010125_highlights_min5.mp4
-rw-r--r-- 1 avidu 197609 1496443316 Jun 13 22:30 KushagraAviHighlight-Trailor.mp4
-rw-r--r-- 1 avidu 197609 125200877 Jun 10 16:31 MahadevpuraSingles_highlights.mp4
-rw-r--r-- 1 avidu 197609      9016 Jun  8 18:24 adarsh_avi_full_vs_human.csv
-rw-r--r-- 1 avidu 197609      561 Jun  9 18:14 adarsh_avi_vs_human.csv
-rw-r--r-- 1 avidu 197609     1721 Jun  6 21:23 adarsh_baseline_gated.csv
-rw-r--r-- 1 avidu 197609     2023 Jun  6 21:15 adarsh_baseline_segments.csv
-rw-r--r-- 1 avidu 197609     1628 Jun  6 22:40 adarsh_gemini_fullvideo.csv
-rw-r--r-- 1 avidu 197609     1670 Jun  6 22:21 adarsh_gemini_refined.csv
-rw-r--r-- 1 avidu 197609      269 Jun  6 22:03 adarsh_gemini_smoke.csv
-rw-r--r-- 1 avidu 197609     1327 Jun  6 21:29 adarsh_min5s_segments.csv
-rw-r--r-- 1 avidu 197609     2895 Jun  6 20:54 adarsh_tuned_segments.csv
-rw-r--r-- 1 avidu 197609     36864 May 19 23:07 badminton_indexer.db
-rw-r--r-- 1 avidu 197609 3979353 Jun 11 20:43 e2e_reel.mp4
-rw-r--r-- 1 avidu 197609     2481 Jun 11 20:43 e2e_server_log.txt
-rw-r--r-- 1 avidu 197609     5645 Jun  8 21:56 gbaaddy_vs_human.csv
-rw-r--r-- 1 avidu 197609     6756 Jun 13 20:19 gx010125_min5_stitch.log

```

```
-rw-r--r-- 1 avidu 197609      108744 Jun 13 15:03 gx010125_run.log
-rw-r--r-- 1 avidu 197609      35165 Jun 12 22:29 gx010125_transcode.log
-rw-r--r-- 1 avidu 197609       1766 Jun 10 12:56 human_lovo_manifest.json
-rw-r--r-- 1 avidu 197609    160570312 Jun 10 16:46 kushagra_1280.mp4
-rw-r--r-- 1 avidu 197609      2659 Jun 10 17:01 kushagra_1280.report.json
-rw-r--r-- 1 avidu 197609      1166 Jun 10 17:01 kushagra_1280.report.md
-rw-r--r-- 1 avidu 197609       480 Jun 10 17:01 kushagra_1280.summary.md
-rw-r--r-- 1 avidu 197609      3200 Jun  8 18:42 kushagra_vs_human.csv
```

31. assistant (2026-06-13T19:55:57.868Z)

Perfect. Let me now look at one of the trajectory CSVs and the rally labels to understand the format better:

32. user (2026-06-13T19:55:58.562Z)

```
Frame,Visibility,X,Y
1,1,1069.8391342163086,489.07470703125
2,1,1069.3811416625977,502.46131896972656
3,1,1071.0103225708008,511.7584419250488
4,1,1073.2082748413086,512.3460388183594
5,1,1073.6650085449219,503.7173080444336
6,1,1076.2910842895508,495.1043701171875
7,1,1075.755615234375,488.3548164367676
8,1,1076.052360534668,480.2399253845215
9,1,1075.8190155029297,473.06450843811035
10,1,1072.859001159668,465.1318073272705
11,1,1076.2835311889648,457.30250358581543
12,1,1076.1379623413086,450.0244617462158
13,1,1078.1643676757812,443.54496002197266
14,1,1079.0762329101562,438.1589698791504
15,1,1079.9838638305664,431.5318965911865
16,1,1080.6429290771484,427.1159076690674
17,1,1081.6316986083984,421.9412040710449
18,1,1083.8132858276367,416.3719940185547
19,1,1085.7146072387695,412.99564361572266
```

33. user (2026-06-13T19:55:58.892Z)

```
rally_number,start_time,end_time,ending_reason,sport
1,37.434,72.194,winner,badminton
2,79.936,112.954,winner,badminton
3,130.193,134.947,winner,badminton
4,139.697,144.944,unforced_error,badminton
5,152.196,157.198,unforced_error,badminton
6,161.694,165.950,unforced_error,badminton
7,170.817,176.327,forced_error,badminton
8,182.829,186.083,unforced_error,badminton
9,193.576,199.831,forced_error,badminton
10,205.832,211.828,unforced_error,badminton
11,221.082,225.827,forced_error,badminton
12,232.832,236.829,forced_error,badminton
13,241.828,246.329,forced_error,badminton
14,252.329,258.080,unforced_error,badminton
15,266.081,271.083,forced_error,badminton
16,276.577,283.578,forced_error,badminton
17,289.830,293.579,unforced_error,badminton
18,300.329,310.079,forced_error,badminton
19,316.829,319.585,unforced_error,badminton
```

34. assistant (2026-06-13T19:56:00.664Z)

Excellent. Now let me check the name-to-file mapping more carefully and look at test files for reference:

35. user (2026-06-13T19:56:01.345Z)

```
1      """Distilled local rally classifier driver (pure parts; no WSL/GPU/Gemini)."""
2      import csv
3
4      from backend.eval import distill_local as dl
5      from backend.eval.classifier import LogisticRegression
6
7
8      def _write_traj(path):
9          rows = [("Frame", "Visibility", "X", "Y")]
10         for i in range(0, 30):
11             rows.append((i, 1, i * 5, 50))
12         for i in range(30, 50):
13             rows.append((i, 0, -1, -1))
14         for i in range(50, 80):
15             rows.append((i, 1, (i - 50) * 5, 50))
16         with open(path, "w", newline="") as f:
17             csv.writer(f).writerows(rows)
18         return str(path)
19
20
21     def test_build_candidates_feature_shape(tmp_path):
22         intervals, X = dl.build_candidates(_write_traj(tmp_path / "t.csv"), fps=10.0,
frame_width=100.0)
23         assert len(intervals) == len(X) >= 1
24         assert all(len(row) == len(dl.FEATURE_NAMES) for row in X) # 5 invariant features
25         assert all(d > 0 for d, *_ in X) # duration positive
26
27
28     def test_predict_rallies_respects_threshold(tmp_path):
29         intervals, X = dl.build_candidates(_write_traj(tmp_path / "t.csv"), fps=10.0,
frame_width=100.0)
30         y = [1] + [0] * (len(intervals) - 1)
31         model = LogisticRegression(epochs=100).fit(X, y, dl.FEATURE_NAMES)
32         none = dl.predict_rallies(model, X, intervals, threshold=1.01) # impossible cutoff
-> nothing
33         assert none == []
34         some = dl.predict_rallies(model, X, intervals, threshold=0.0) # everything passes
-> merged windows
35         assert len(some) >= 1
36
```

36. user (2026-06-13T19:56:01.713Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\tests/test_annotation_providers.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests/test_api_endpoints.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests/test_async_jobs.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests/test_automated_provider.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests/test_base.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests/test_calibrate_local.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests/test_calibrate_wasb.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests/test_calibration.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests/test_cleanup_caches.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests/test_compiler.py
```

37. assistant (2026-06-13T19:56:03.224Z)

Now let me look at the rally_gate module to understand how to extract net_crossings:

38. user (2026-06-13T19:56:03.438Z)

```
backend\detectors\__pycache__\rally_gate.cpython-313.pyc
tests\__pycache__\test_rally_gate.cpython-313.pyc
```

```
tests\__pycache__\test_rally_gate.cpython-313-pytest-9.0.3.pyc
backend\pipeline\detectors\rally_gate.py
tests\test_rally_gate.py
backend\pipeline\detectors\__pycache__\rally_gate.cpython-313.pyc
```

39. user (2026-06-13T19:56:04.781Z)

```
1      """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3      WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4      the opponent for service after losing a point, that single trajectory looks like a
5      rally. The discriminator is structural, not duration-based: a real rally has the
6      shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7      single service feed crosses -once.
8
9      This module is camera-agnostic and needs NO new model ? it works on the trajectory
10     we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11     axis with the larger shuttle spread) and the net position (median shuttle coord on
12     that axis ? the shuttle clusters at the two ends and crosses the net region often,
13     so the median sits near the net), then counts sign changes of (coord - net) across
14     the visible points inside a candidate window, with a deadband to ignore jitter.
15
16     Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17     Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18     windows from trajectory_to_action_windows, or to fold into it later.
19     """
20     from __future__ import annotations
21
22     from dataclasses import dataclass
23     from typing import List, Optional, Sequence, Tuple
24
25     Interval = Tuple[float, float]
26
27
28     @dataclass
29     class GatedWindow:
30         start: float
31         end: float
32         crossings: int
33         visible_points: int
34         kept: bool
35
36
37     def _spread(vals: Sequence[float]) -> float:
38         """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39         if len(vals) < 2:
40             return 0.0
41         s = sorted(vals)
42         lo = s[max(0, int(0.05 * (len(s) - 1)))]
43         hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
44         return hi - lo
45
46
47     def _median(vals: Sequence[float]) -> float:
48         if not vals:
49             return 0.0
50         s = sorted(vals)
51         n = len(s)
52         return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])
53
54
55     def estimate_play_axis(points) -> Tuple[str, float, float]:
56         """Return (axis 'x'|'y', net_value, span) from visible points.
57
58         Play axis = the image axis along which the shuttle travels most (larger spread).
```

```

59     For a baseline camera that's Y (players top/bottom); for a side camera, X.
60     """
61     xs = [p.x for p in points if p.visible]
62     ys = [p.y for p in points if p.visible]
63     sx, sy = _spread(xs), _spread(ys)
64     if sx >= sy:
65         return "x", _median(xs), sx
66     return "y", _median(ys), sy
67
68
69 def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
70     """Count sign changes of (coord - net) across visible points, ignoring the
71     deadband zone near the net (so jitter at the net doesn't inflate the count)."""
72     crossings = 0
73     last_side = None
74     for p in window_points:
75         if not p.visible:
76             continue
77         v = (p.x if axis == "x" else p.y) - net
78         if abs(v) < deadband:
79             continue
80         side = v > 0
81         if last_side is not None and side != last_side:
82             crossings += 1
83         last_side = side
84     return crossings
85
86
87 def _points_in(points, fps: float, start: float, end: float):
88     return [p for p in points if start * fps <= p.frame <= end * fps]
89
90
91 def gate_windows(points, windows: List[Interval], fps: float,
92                 min_crossings: int = 2, deadband_frac: float = 0.06,
93                 estimate: Optional[Tuple[str, float, float]] = None) ->
List[GatedWindow]:
94     """Annotate each (start,end) window with its shuttle net-crossing count and a
95     keep decision (crossings >= min_crossings). Deadband is a fraction of the play
96     axis span. Net/axis are estimated once over the whole trajectory (stable).
97
98     `estimate` lets a caller scoring MANY windows off one trajectory pass a precomputed
99     `(axis, net, span)` (from `estimate_play_axis`) so the  $O(P \log P)$  estimate isn't
redone
100     per window ? same result, avoids the per-window re-estimation."""
101     axis, net, span = estimate if estimate is not None else estimate_play_axis(points)
102     deadband = deadband_frac * span
103     out: List[GatedWindow] = []
104     for (s, e) in windows:
105         wp = _points_in(points, fps, s, e)
106         c = count_net_crossings(wp, axis, net, deadband)
107         vis = sum(1 for p in wp if p.visible)
108         out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
109                               kept=c >= min_crossings))
110     return out
111
112
113 def apply_gate(points, windows: List[Interval], fps: float,
114               min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
115     """Convenience: return only the windows that pass the crossing gate."""
116     return [(g.start, g.end) for g in
117             gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
118

```

Perfect! Now I have all the necessary information. Let me create a comprehensive design document:

41. user (2026-06-13T19:56:48.407Z)

Output does not match required schema: root: must have required property 'findings', root: must have required property 'anchors'

42. assistant (2026-06-13T19:56:50.763Z)

Now let me provide the final design document with proper structured output:

43. user (2026-06-13T19:57:24.995Z)

Output does not match required schema: root: must have required property 'findings', root: must have required property 'anchors'

44. assistant (2026-06-13T19:58:08.181Z)

Let me provide the findings directly:

OFFLINE EXTRACTION OF PERSON FEATURES FOR GOLDEN VIDEOS ? DESIGN

Name-to-File Mapping (Golden Videos)

The manifest (`output/human_lovo_manifest.json`) lists 6 golden videos by internal `name` field. Map to video files in `C:/Users/avidu/Projects/Annotation Setup/Collect/Golden Labelled/`:

```
| Manifest `name` | Video File | FPS | Frame Width | Labels CSV |
|---|---|---|---|---|
| `adarsh_avi_singles` | `Adarsh and Avi.MP4` | 59.94 | 1920 | `Adarsh and Avi.rallies.csv` |
| `kushagra_singles` | `KushagraYashAviFirstVideo.MP4` | 59.94 | 1920 |
`KushagraYashAviFirstVideo.rallies.csv` |
| `largetest_doubles` | `LargeTestVideo.MP4` | 29.97 | 1920 | `LargeTestVideo.rallies.csv` |
| `gbaaddy` | `gBaaddy20May2026.MP4` | 59.94 | 1920 | `gBaaddy20May2026.rallies.csv` |
| `testlarge_short` | `TestLargeVideo.MP4` | 29.97 | 1920 | `TestLargeVideo.rallies.csv` |
| `mahadevpura_singles` | `MahadevpuraSingles.MP4` | 29.97 | 1920 |
`MahadevpuraSingles.rallies.csv` |
```

Manifest Extension Spec

Extend `output/human_lovo_manifest.json` to add the `video_path` field per entry:

```
```json
[
 {
 "name": "adarsh_avi_singles",
 "video_path": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden Labelled/Adarsh and
Avi.MP4",
 "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/adarsh_avi_full_traj.csv",
 "fps": 59.94,
 "frame_width": 1920,
 "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden Labelled/Adarsh and
Avi.rallies.csv"
 },
 ...
]
```

The driver reads each entry's `fps`, `frame_width`, `labels`, and **new** `video_path` field.



---

## Golden Feature Extraction Pipeline

**\*\*Input per golden video:\*\*** trajectory CSV, video path, FPS, frame\_width, golden labels CSV

**\*\*Output per golden video:\*\*** labeled dataset `{name}\_golden\_features.json` with:

```
```json
{
  "name": "adarsh_avi_singles",
  "num_candidates": 42,
  "candidates": [
    {
      "start_frame": 2245,
      "end_frame": 4333,
      "start_time": 37.434,
      "end_time": 72.194,
      "shuttle_features": {
        "duration": 34.76,
        "peak_velocity": 0.087,
        "mean_velocity": 0.042,
        "active_frame_count": 1823,
        "net_crossings": 14
      },
      "person_features": {
        "players_in_court": 2.0,
        "two_sided_motion": 0.87,
        "mean_player_motion": 0.031,
        "in_court_ratio": 0.92,
        "shuttle_player_colocation": 0.18
      },
      "label": 1,
      "label_source": "golden_csv"
    },
    ...
  ]
}
```
```

---

## Step-by-Step Driver Algorithm

**\*\*Per golden video:\*\***

1. **\*\*Load manifest entry\*\*** ? extract `fps`, `frame\_width`, `trajectory`, `video\_path`, `labels`

2. **\*\*Build shuttle candidates\*\*** (recall-oriented proposal):

```
```python
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
# Proposal thresholds (distill_local.PROPOSAL = (0.005, 1.0, 0.5))
windows = TrackNetRunner.trajectory_to_action_windows(
    points, fps=fps, frame_width=frame_width,
    velocity_thresh=0.005,      # very permissive
    merge_gap=1.0,
    min_window_duration=0.5
)
```
```

**\*\*References:\*\***

```
- `backend/pipeline/detectors/tracknet_runner.py:252-319` ? `trajectory_to_action_windows()`
- `backend/eval/distill_local.py:47-68` ? `build_candidates()` feature extraction
```

3. **\*\*Enrich candidates with shuttle features\*\*** (from `distill\_local.build\_candidates`):

```
```python
from backend.pipeline.detectors import rally_gate
```

```

from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

intervals = [(w["start"], w["end"]) for w in windows]
gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0)

shuttle_features = []
for w, g in zip(windows, gated):
    dur = max(1e-6, w["end"] - w["start"])
    win_frames = max(1.0, dur * fps)
    shuttle_features.append({
        "duration": dur,
        "peak_velocity": float(w["peak_velocity"]),
        "mean_velocity": float(w["mean_velocity"]),
        "active_frame_count": float(w["active_frame_count"]),
        "net_crossings": float(g.crossings) # Gate A discriminator
    })
...

**References:**
- `backend/eval/distill_local.py:40-68` ? FEATURE_NAMES, build_candidates logic
- `backend/pipeline/detectors/rally_gate.py:28-117` ? `gate_windows()` for net-crossing count

```

4. ****Per window, extract frame bounds and run PersonDetector**:**

```

```python
from backend.pipeline.detectors.person_detector import PersonDetector
from backend.config import load_config # or Config()

config = load_config() # read config.json
detector = PersonDetector(config)
detector.lazy_load_model()

for w in windows:
 start_time, end_time = w["start"], w["end"]
 start_frame = int(start_time * fps)
 end_frame = int(end_time * fps)

 result = detector.detections_per_frame(
 video_path=video_path,
 start_frame=start_frame,
 end_frame=end_frame,
 frame_skip=3 # every 3rd frame
)
 per_frame = result["frames"] # {frame_idx: [(track_id, bbox), ...]}
...

References:
- `backend/pipeline/detectors/person_detector.py:258-303` ? `detections_per_frame()`
signature & contract
- Returns: `{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
"frame_height"}`
- Frame indices **align with trajectory CSV** (original video frame numbering)

```

5. **\*\*Extract person features per window\*\*:**

```

```python
from backend.pipeline.detectors.fusion_features import compute_person_features,
PERSON_FEATURE_NAMES

# Load golden labels
gts = training.label_candidates.load_golden_gts(labels_csv)
# or manually parse:
gts = []
with open(labels_csv) as f:
    reader = csv.DictReader(f)
    for row in reader:
        gts.append((float(row["start_time"]), float(row["end_time"])))

# Label each candidate against golden rallies

```

```

from backend.eval.training import label_candidates
y = label_candidates(intervals, gts, iou_threshold=0.5)

# Per window extract person features
person_features_list = []
for w, per_frame in zip(windows, all_per_frame_results):
    shuttle_points = [p for p in points if start_frame <= p.frame <= end_frame]
    pf = compute_person_features(
        per_frame=per_frame,
        shuttle_points=shuttle_points,
        frame_width=frame_width,
        frame_height=result["frame_height"], # from detections_per_frame result
        court_polygon=None, # no court mask available
        net=None, # no net available
        min_players=2
    )
    person_features_list.append(pf)
...

**References:**
- `backend/pipeline/detectors/fusion_features.py:200-215` ? `compute_person_features()`
signature
- Returns dict with keys: `["players_in_court", "two_sided_motion", "mean_player_motion",
"in_court_ratio", "shuttle_player_colocation"]`
- `backend/pipeline/detectors/fusion_features.py:194-197` ? `PERSON_FEATURE_NAMES`
- `backend/eval/training.py:50-61` ? `label_candidates()` for y-labeling

6. **Persist per-video labeled dataset**:
```python
output = {
 "name": manifest["name"],
 "num_candidates": len(windows),
 "candidates": [
 {
 "start_frame": int(w["start"] * fps),
 "end_frame": int(w["end"] * fps),
 "start_time": w["start"],
 "end_time": w["end"],
 "shuttle_features": shuttle_features[i],
 "person_features": person_features_list[i],
 "label": y[i],
 "label_source": "golden_csv"
 }
 for i, w in enumerate(windows)
]
}
with open(f"output/{name}_golden_features.json", "w") as f:
 json.dump(output, f, indent=2)
...

```

---

## Key Data Structure Alignments

### \*\*Frame Index Convention:\*\*

- All frame indices are from the **original full video** (not re-extracted clips).
- `TrajectoryPoint.frame` from `tracknet\_runner.py:80-84` ? raw video frame numbers.
- `PersonDetector.detections\_per\_frame()` reads by absolute frame index (seek + sequential read); returned indices match trajectory.
- **Shuttle trajectory and person detections share the same frame indexing** ? critical for `fusion\_features.shuttle\_player\_colocation()` to work.

### \*\*Bounding Box Format:\*\*

- Person boxes: `(x1, y1, x2, y2)` in **pixels** (top-left, bottom-right).
- Shuttle points: `(x, y)` in **pixels**.
- All normalized coordinates in `fusion\_features.py` functions are 0-1 (divide by

```
frame_width/frame_height).
```

```
Label Matching:
```

```
- `training.label_candidates()` performs IoU-based matching between candidate windows and golden
GT intervals (same matcher used in evaluation/scoring).
- Golden labels CSV: `start_time`, `end_time` in seconds.
- Candidates: `start_frame`, `end_frame` from trajectory windows ? convert to time via `frame /
fps`.
```

```

```

## Experiment Harness Integration

The persisted labeled dataset feeds directly into the harness:

```
```python  
from backend.eval.experiment import VideoCandidates, Variant, evaluate_variant
```

Load persisted golden features

```
with open(f"output/{name}_golden_features.json") as f:  
    data = json.load(f)
```

Reconstruct VideoCandidates for experiment.py

```
intervals = [(c["start_time"], c["end_time"]) for c in data["candidates"]]  
features = {  
    "duration": [c["shuttle_features"]["duration"] for c in data["candidates"]],  
    "peak_velocity": [c["shuttle_features"]["peak_velocity"] for c in data["candidates"]],  
    "mean_velocity": [c["shuttle_features"]["mean_velocity"] for c in data["candidates"]],  
    "active_frame_count": [c["shuttle_features"]["active_frame_count"] for c in  
data["candidates"]],  
    "net_crossings": [c["shuttle_features"]["net_crossings"] for c in data["candidates"]],  
    # NEW: person features  
    "players_in_court": [c["person_features"]["players_in_court"] for c in data["candidates"]],  
    "two_sided_motion": [c["person_features"]["two_sided_motion"] for c in data["candidates"]],  
    "mean_player_motion": [c["person_features"]["mean_player_motion"] for c in  
data["candidates"]],  
    "in_court_ratio": [c["person_features"]["in_court_ratio"] for c in data["candidates"]],  
    "shuttle_player_colocation": [c["person_features"]["shuttle_player_colocation"] for c in  
data["candidates"]],  
}  
gts = [(start, end) for start, end in intervals if matched_to_golden] # per y[i] == 1  
  
vc = VideoCandidates(name=data["name"], intervals=intervals, features=features, gts=gts)
```

Score with a learned variant using person features

```
variant = Variant(  
    name="shuttle+person",  
    feature_names=[  
        "duration", "peak_velocity", "mean_velocity", "active_frame_count", "net_crossings",  
        "players_in_court", "two_sided_motion", "mean_player_motion", "in_court_ratio",  
        "shuttle_player_colocation"  
    ],  
    min_crossings=2 # Gate A  
)  
result = evaluate_variant([vc], variant)  
```
```

```
References:
```

```
- `backend/eval/experiment.py:154-168` ? `VideoCandidates` dataclass
- `backend/eval/experiment.py:203-224` ? `predict()` pure offline scoring
- `backend/eval/experiment.py:245-300` ? `evaluate_variant()` + LOVO harness
```

```

```

## Critical Files & Line References

```
Module	File	Key Functions / Structures
Trajectory ? Candidates	`backend/pipeline/detectors/tracknet_runner.py`	
`parse_trajectory_csv` (233), `trajectory_to_action_windows` (252?319)		
Shuttle Features	`backend/eval/distill_local.py`	`build_candidates` (47?68),
`FEATURE_NAMES` (40)		
Rally Gate (Net Crossings)	`backend/pipeline/detectors/rally_gate.py`	`gate_windows`
(91?110), `estimate_play_axis` (55?66), `count_net_crossings` (69?84)		
Person Detection	`backend/pipeline/detectors/person_detector.py`	
`PersonDetector.detections_per_frame` (258?303), `detect_and_track` (100?138)		
Person Features	`backend/pipeline/detectors/fusion_features.py`	
`compute_person_features` (200?215), `PERSON_FEATURE_NAMES` (194?197)		
Golden Labeling	`backend/eval/training.py`	`label_candidates` (50?61)
Experiment Harness	`backend/eval/experiment.py`	`VideoCandidates` (154?168), `Variant`
(73?143), `predict` (203?224), `evaluate_variant` (245?300) |
```

---

## Implementation Notes

1. **Lazy Model Loading:** `PersonDetector.lazy\_load\_model()` loads the torchvision detector on first call. GPU availability checked at line `backend/pipeline/detectors/person\_detector.py:75`.
2. **ByteTrack per-window:** A fresh tracker is created per video (not per-window) to maintain track-ID consistency within the entire video; this is already handled by `detections\_per\_frame()`.
3. **Court Polygon / Net:** Not available for golden videos; pass `court\_polygon=None` and `net=None` to `compute\_person\_features()`. Features degrade gracefully (fusion\_features.py lines 30?77).
4. **Frame-Skip in Person Detection:** `detections\_per\_frame` uses `frame\_skip=3` (every 3rd frame). Adjust if needed; trajectory alignment is **frame-based**, so skipped frames still map to the correct trajectory index.
5. **Recall-Oriented Proposal:** `PROPOSAL = (0.005, 1.0, 0.5)` from `distill\_local.py:43` is intentionally permissive (low velocity threshold). This generates many candidates; the golden labels filter true positives for training.
6. **Golden Labels Format:** Rally CSV has columns `rally\_number, start\_time, end\_time, ending\_reason, sport`. Parse only `start\_time` and `end\_time` (in seconds).

---

```
<StructuredOutput>
 <findings>
```

## Offline Feature Extraction Pipeline for Golden Videos (P3 GPU Step)

**Manifest Extension:** Add `video\_path` field to `output/human\_lovo\_manifest.json` per entry, mapping 6 golden videos:

```
- `adarsh_avi_singles` ? `C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/Adarsh and Avi.MP4`
- `kushagra_singles` ? `KushagraYashAviFirstVideo.MP4`
- `largetest_doubles` ? `LargeTestVideo.MP4`
- `gbaaddy` ? `gBaaddy20May2026.MP4`
- `testlarge_short` ? `TestLargeVideo.MP4`
- `mahadevpura_singles` ? `MahadevpuraSingles.MP4`
```

**Per-Video Driver (6 steps):**

1. Parse manifest + trajectory CSV; build recall-oriented shuttle candidates via `TrackNetRunner.trajectory\_to\_action\_windows(points, fps, frame\_width, velocity\_thresh=0.005, merge\_gap=1.0, min\_window\_duration=0.5)` (distill\_local.PROPOSAL)

```

2. Enrich with shuttle features: `duration`, `peak_velocity`, `mean_velocity`,
`active_frame_count`, `net_crossings` (via `rally_gate.gate_windows()`)
3. Per window, run `PersonDetector.detections_per_frame(video_path, start_frame, end_frame,
frame_skip=3)` over original video frames to get frame-aligned detections
4. Compute person features via `fusion_features.compute_person_features(per_frame,
shuttle_points, frame_width, frame_height, court_polygon=None, net=None)` ?
`["players_in_court", "two_sided_motion", "mean_player_motion", "in_court_ratio",
"shuttle_player_colocation"]`
5. Label y-values via `training.label_candidates(intervals, gts, iou=0.5)` from golden CSV
(start_time, end_time)
6. Persist `{name}_golden_features.json` with candidates array: `[{start_frame, end_frame,
start_time, end_time, shuttle_features, person_features, label, label_source}]`

Output Schema: Per-video `output/{name}_golden_features.json` contains labeled dataset ready
for `experiment.py` harness via `VideoCandidates(name, intervals, features={feature_name: [per-
candidate values]}, gts=[...])`.

Frame Alignment Critical: `PersonDetector.detections_per_frame()` reads by absolute frame
index (seek + sequential, no re-extraction); returned frames align with trajectory CSV's frame
numbering. Enables `fusion_features.shuttle_player_colocation()` and `two_sided_motion()`.

Experiment Harness Ready: Persisted features feed `experiment.py:Variant` with
`feature_names=[shuttle_features + person_features]` for LOVO training via
`evaluate_variant(videos, variant, honest=True)`.
</findings>
<anchors>
- backend/pipeline/detectors/tracknet_runner.py:252?319 (trajectory_to_action_windows)
- backend/pipeline/detectors/tracknet_runner.py:233?249 (parse_trajectory_csv)
- backend/eval/distill_local.py:40?68 (FEATURE_NAMES, build_candidates)
- backend/pipeline/detectors/rally_gate.py:91?110 (gate_windows for net_crossings)
- backend/pipeline/detectors/person_detector.py:258?303 (detections_per_frame)
- backend/pipeline/detectors/person_detector.py:100?138 (detect_and_track)
- backend/pipeline/detectors/fusion_features.py:200?215 (compute_person_features)
- backend/pipeline/detectors/fusion_features.py:194?197 (PERSON_FEATURE_NAMES)
- backend/eval/training.py:50?61 (label_candidates)
- backend/eval/experiment.py:154?168 (VideoCandidates)
- backend/eval/experiment.py:73?143 (Variant)
- backend/eval/experiment.py:203?224 (predict)
- backend/eval/experiment.py:245?300 (evaluate_variant + LOVO)
- C:\Users\avidu\Projects\badminton-highlight-indexer\output\human_lovo_manifest.json
(manifest location)
- C:\Users\avidu\Projects\Annotation Setup\Collect\Golden Labelled\ (video directory)
- C:\Users\avidu\Projects\Annotation Setup\Collect\Trajectories\ (trajectory CSV directory)
</anchors>
</StructuredOutput>

```

#### 45. user (2026-06-13T19:58:08.225Z)

Stop hook feedback:

You did not call StructuredOutput. You MUST call StructuredOutput to return your answer ? the tool input IS your answer. Call it now.

#### 46. user (2026-06-13T19:58:19.775Z)

Output does not match required schema: root: must have required property 'findings', root: must have required property 'anchors'

#### 47. assistant (2026-06-13T19:58:21.348Z)

I need to provide the required parameters. Let me call it correctly:

#### 48. user (2026-06-13T19:58:30.769Z)

Structured output provided successfully

#### 49. assistant (2026-06-13T19:58:37.293Z)

Perfect. I have provided a complete design specification for the offline extraction of PERSON features for the 6 golden videos (P3 GPU step). Here's the executive summary:

### Design Summary

**\*\*Manifest Extension:\*\*** The existing `output/human_lovo_manifest.json` needs a `video_path` field added to each entry, mapping the internal `name` field to the actual MP4 files in `C:/Users/avidu/Projects/Annotation Setup/Collect/Golden Labelled/`.

**\*\*Per-Video Driver Pipeline (6 Steps):\*\***

1. Load trajectory CSV and build recall-oriented shuttle candidates using `TrackNetRunner.trajectory_to_action_windows()` with proposal thresholds (`velocity_thresh=0.005`, `merge_gap=1.0`, `min_window_duration=0.5`)
2. Enrich candidates with shuttle features (`duration`, `peak_velocity`, `mean_velocity`, `active_frame_count`, `net_crossings`) via `rally_gate.gate_windows()`
3. Run `PersonDetector.detections_per_frame()` over original video frames for each candidate window to get frame-aligned person detections
4. Compute ~5 person features per window using `fusion_features.compute_person_features()` with detected boxes and shuttle trajectory
5. Label each candidate (`y=0/1`) using `training.label_candidates()` against golden CSV rallies via IoU matching (`iou=0.5`)
6. Persist per-video labeled dataset as `{name}_golden_features.json` with shuttle+person features and labels

**\*\*Critical Data Alignment:\*\*** `PersonDetector.detections_per_frame()` reads the video by absolute frame index and returns frame indices that align directly with the trajectory CSV's frame numbering, enabling correct computation of features like `shuttle_player_colocation()` and `two_sided_motion()`.

**\*\*Harness Integration:\*\*** The persisted features reconstruct `VideoCandidates` objects that feed directly into `experiment.py` for Variant-based offline re-scoring and honest LOVO evaluation.